

PRACTICAL WEEK 5

1. Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical5.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define DATA_SIZE 100000

int main()
{
    int pipefd[2];
    pid_t pid;
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    pid = fork();

    if (pid == -1)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid == 0)
    {
        close(pipefd[1]);

        int value;
        long count = 0;
        long sum = 0;

        while (read(pipefd[0], &value, sizeof(value)) > 0)
        {
            sum += value;
            count++;
        }

        close(pipefd[0]);

        printf("Consumer: Received %ld values\n", count);
        printf("Consumer: Sum of values = %ld\n", sum);

        exit(EXIT_SUCCESS);
    }
```

```

else
{
    close(pipefd[0]);

    clock_t start, end;

    start = clock();

    for (int i = 1; i <= DATA_SIZE; i++)
    {
        if (write(pipefd[1], &i, sizeof(i)) == -1)
        {
            perror("write");
            exit(EXIT_FAILURE);
        }
    }

    close(pipefd[1]);

    wait(NULL);

    end = clock();

    double time_taken =
        ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\nProducer: Sent %d values\n", DATA_SIZE);
    printf("Communication time = %.6f seconds\n", time_taken);

    if (time_taken > 0)
    {
        double rate =
            DATA_SIZE / time_taken;

        printf("Communication rate = %.2f values/second\n",
            rate);
    }
}

return 0;
}

```

Run the code:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical5.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical5.1.c -o practical5.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical5.1

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical5.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical5.1.c -o practical5.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical5.1
Consumer: Received 100000 values
Consumer: Sum of values = 5000050000

Producer: Sent 100000 values
Communication time = 0.028366 seconds
Communication rate = 3525347.25 values/second
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

2. Develop a program that executes the equivalent of the shell command: `ls-1 | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical5.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    pid1 = fork();

    if (pid1 == -1)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid1 == 0)
    {
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("ls", "ls", "-1", (char *)NULL);

        perror("execlp ls");
        exit(EXIT_FAILURE);
    }

    pid2 = fork();
```

```

    if (pid2 == -1)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid2 == 0)
    {
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        execlp("grep", "grep", ".c", (char *)NULL);

        perror("execlp grep");
        exit(EXIT_FAILURE);
    }

    close(pipefd[0]);
    close(pipefd[1]);
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("\nCommand execution completed.\n");

    return 0;
}

```

Run the code:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical5.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical5.2.c -o practical5.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical5.2

```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical5.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical5.2.c -o practical5.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical5.2
practical2.1
practical2.1.c
practical2.1.txt
practical3.1
practical3.1.c
practical3.2
practical3.2.c
practical4.1
practical4.1.c
practical4.2
practical4.2.c
practical5.1
practical5.1.c
practical5.2
practical5.2.c
practical_fixed
practical_fixed.c
problem1.c
skill1.1.c
skill1.2.c
skill1.3.c
skill2.1.c
skill2.2.c
skill2.3.c
skill2.4.c
skill2.c
skill3.1.c
skill3.2.c
skill3.3.c
skill3.4.c
skill3.c
skill3main2.c
skill4.1.c
skill4.2.c
skill4.3.c
skill4.4.c
source.txt
sum1.c
task2.c
vishwa.c

Command execution completed.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```